

# Session 18

## Backpropagation in NN

**Deep Learning | Zahra Amini**

Telegram: @zahraamini\_ai & Instagram:@zahraamini\_ai & LinkedIn: @zahraamini-ai

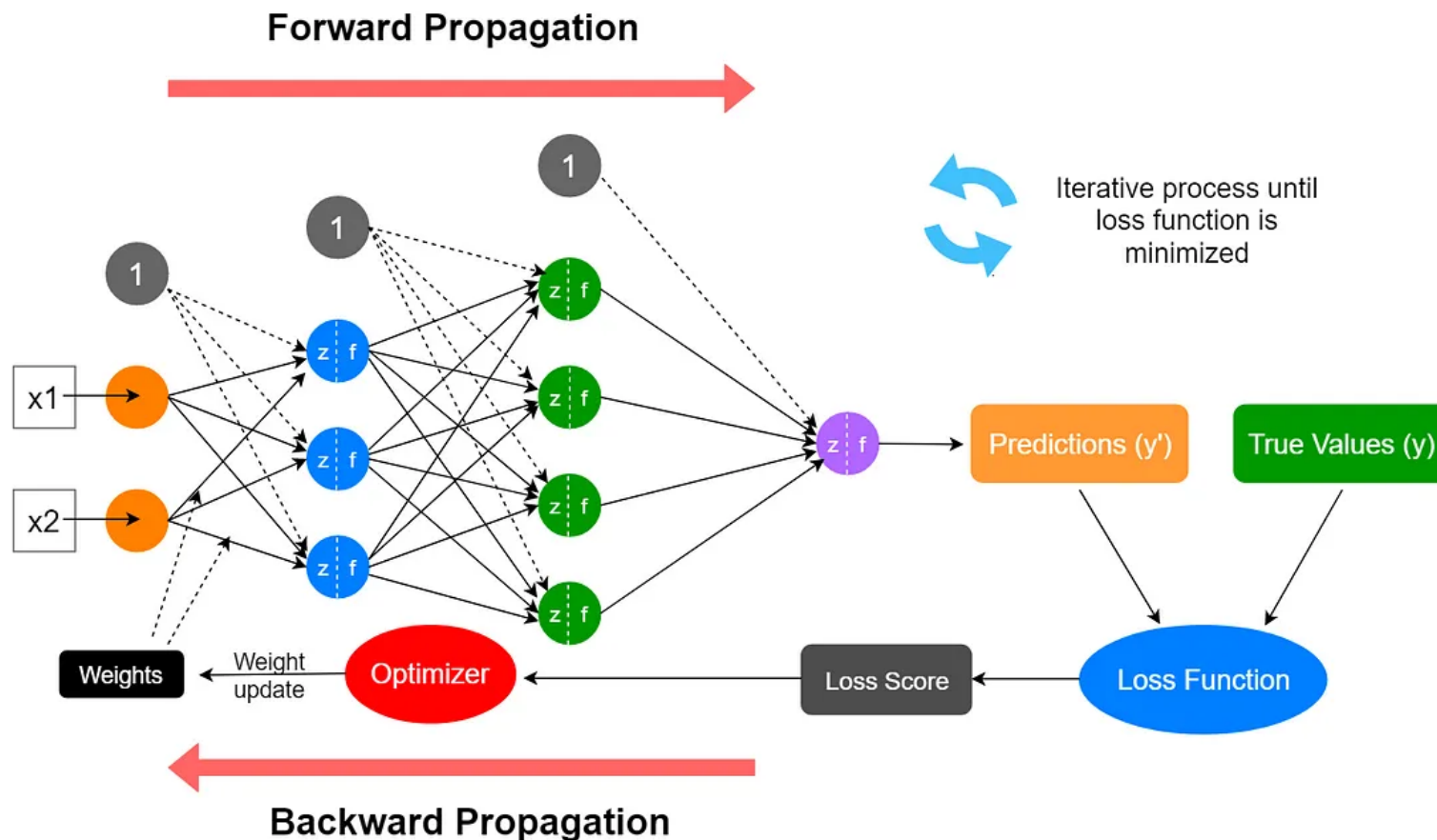
<https://zil.ink/zahraamini>

یادگیری در نورون مصنوعی چیست؟

یادگیری در نورون مصنوعی فرآیندی است که در آن مدل با تنظیم پارامترهای داخلی خود از

طریق به روزرسانی وزن ها و بهینه سازی تابع خطا، توانایی تشخیص و پیش بینی الگوها در

داده های ورودی را به دست می آورد



# Gradient Descent

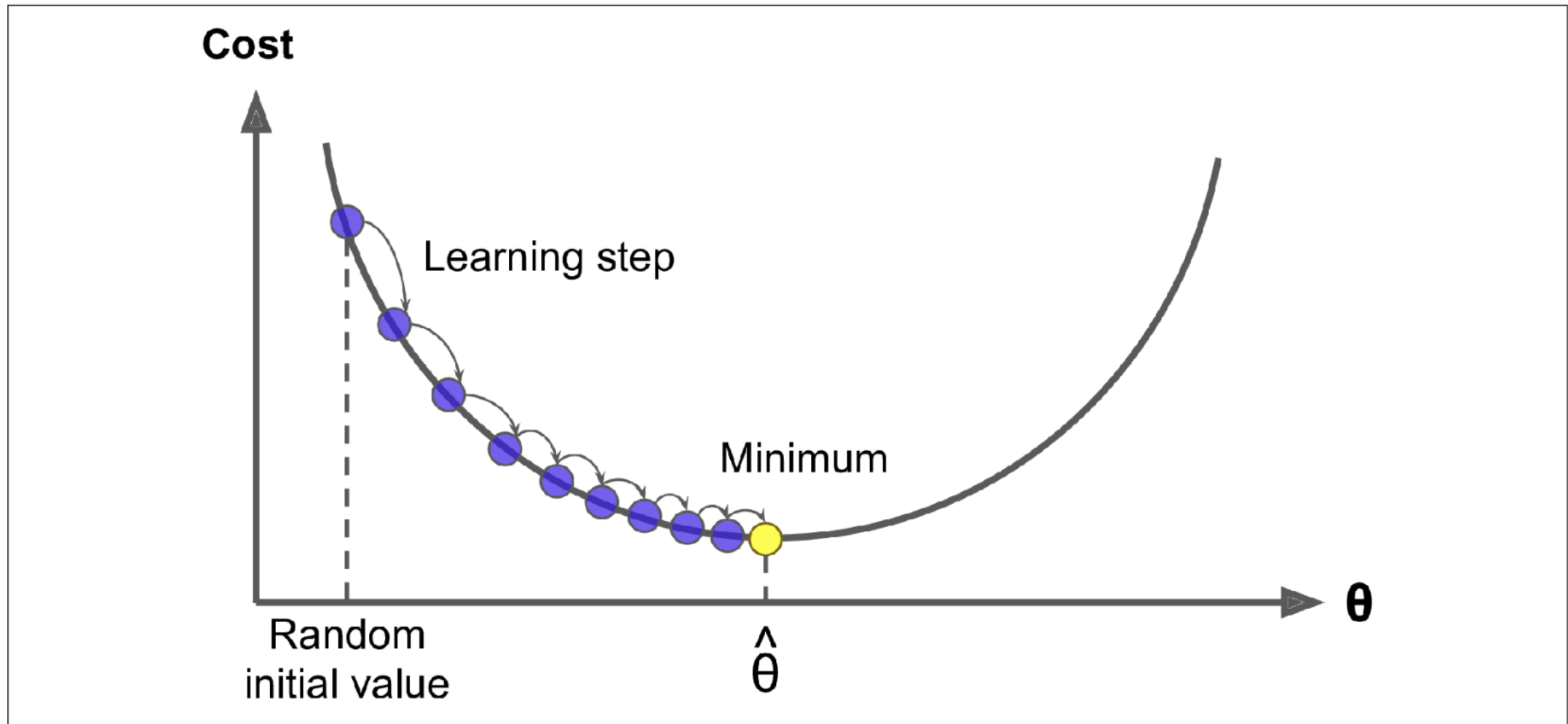


Figure 4-3. In this depiction of Gradient Descent, the model parameters are initialized randomly and get tweaked repeatedly to minimize the cost function; the learning step size is proportional to the slope of the cost function, so the steps gradually get smaller as the parameters approach the minimum

# Learning Rate

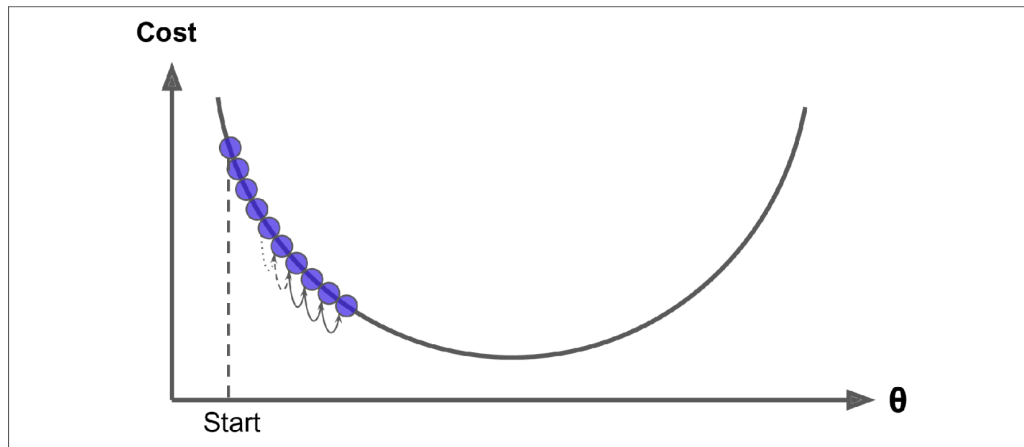


Figure 4-4. The learning rate is too small

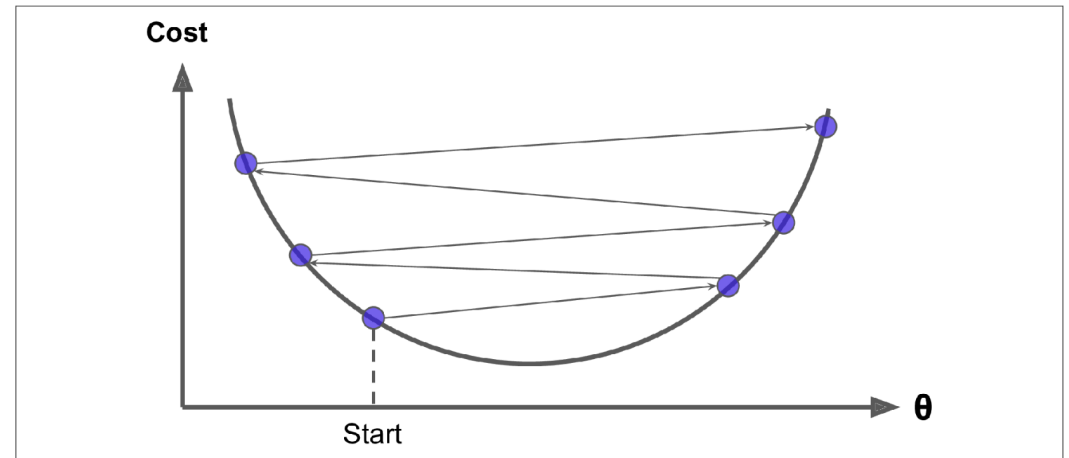
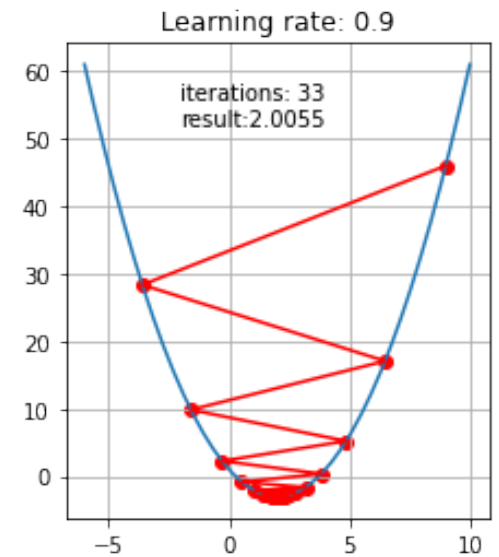
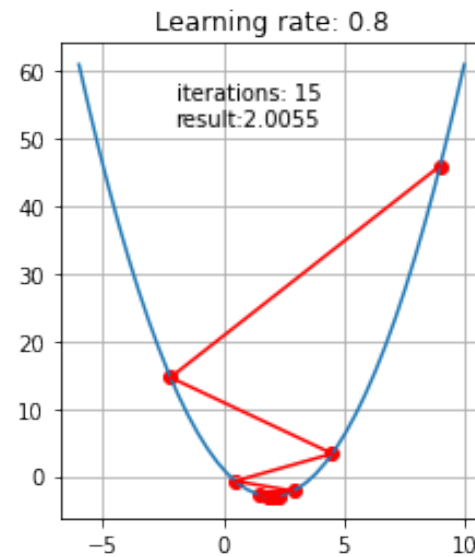
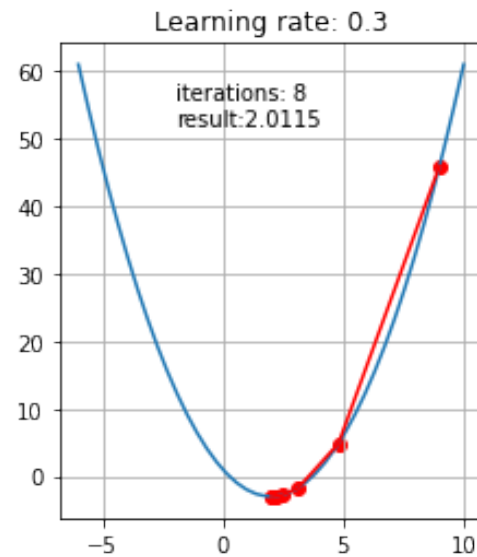
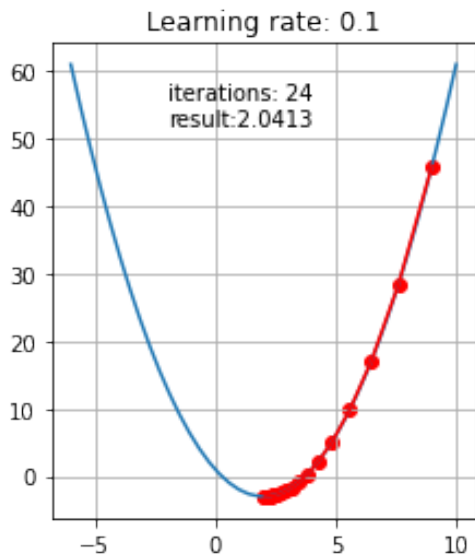


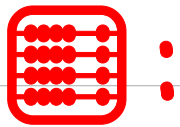
Figure 4-5. The learning rate is too large

\* Convex





# Gradient



گرادیان در ریاضیات و به ویژه در زمینه بهینه سازی و یادگیری ماشین، نشان دهنده شیب یا نرخ تغییرات یک تابع در یک نقطه خاص است. به عبارت ساده تر، گرادیان برداری است که نشان می دهد چگونه مقدار تابع در یک نقطه خاص تغییر می کند و در چه جهتی این تغییرات بیشترین مقدار را دارد

$$\text{Gradient} \leadsto \nabla f(x, y) = \frac{\partial f}{\partial x} \hat{x} + \frac{\partial f}{\partial y} \hat{y}$$

$$f(x, y) = x^2 + y^2 \leadsto \nabla f = 2x \hat{x} + 2y \hat{y}$$

$$\text{مشتق جزئی} \leadsto f(x, y) = x^2 + y^2 \begin{cases} \leadsto \frac{\partial f}{\partial x} = 2x \\ \leadsto \frac{\partial f}{\partial y} = 2y \end{cases}$$

$$\begin{cases} y = 8z^3 + z^2 + 5 \\ z = x^5 + 9 \end{cases} \leadsto \frac{\partial y}{\partial x} = ? \leadsto \text{مشتق زنجیره ای}$$

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial z} \frac{\partial z}{\partial x}$$

# Gradient Descent

① مقداردهی اولیه پارامترها ←  $W$  ها را به صورت رندوم مقداردهی می کنیم

② محاسبه گرادیان ← از تابع Loss نسبت به  $W$  مشتق می گیریم.

③ آپدیت وزها ← 
$$W_{new} = W_{old} - \alpha \frac{\partial L}{\partial W_{old}}$$

④ شرط خاتمه ← مراحل 2 و 3 را تا رسیدن به Minimum یا Iteration مشخصه ادامه می دهیم

# Backpropagation in MLP



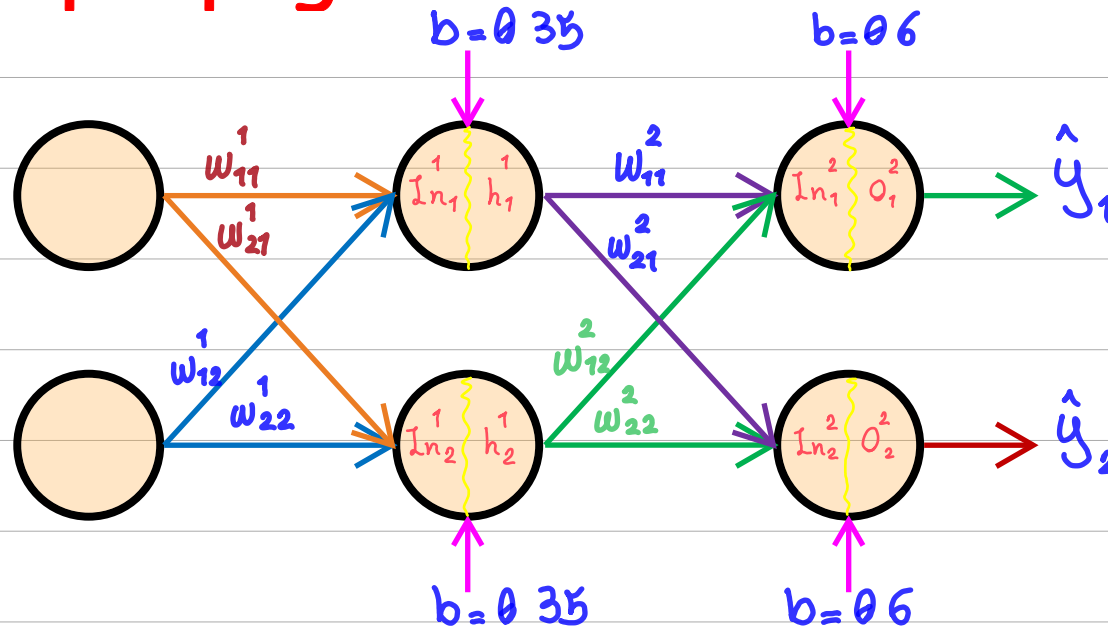
$$x_1 = 0.05, x_2 = 0.1$$

$$W^1 = \begin{bmatrix} 0.15 & 0.2 \\ 0.25 & 0.3 \end{bmatrix} \quad W^2 = \begin{bmatrix} 0.4 & 0.45 \\ 0.5 & 0.55 \end{bmatrix}$$

$$b^1 = 0.35, b^2 = 0.6 \quad y_1 = 0.01, y_2 = 0.99$$

Activation Function  $f^1 = \text{ReLU}$   $f^2 = \text{Sigmoid}$

? Update  $W_{11}^1 \rightsquigarrow$  SGD  $\alpha = 0.1$



- ① Feed Forward
- ② Loss Function
- ③ Back Propagation

① Feed Forward:

$$In_1^1 = W_{11}^1 x_1 + W_{12}^1 x_2 + b^1 = 0.3375 \rightarrow h_1^1 = \text{ReLU}(In_1^1) = 0.3375$$

$$In_2^1 = W_{21}^1 x_1 + W_{22}^1 x_2 + b^1 = 0.5675 \rightarrow h_2^1 = \text{ReLU}(In_2^1) = 0.5675$$

$$In_1^2 = W_{11}^2 h_1^1 + W_{12}^2 h_2^1 + b^2 = 1.1057 \rightarrow O_1^2 = \text{Sigmoid}(In_1^2) = \hat{y}_1$$

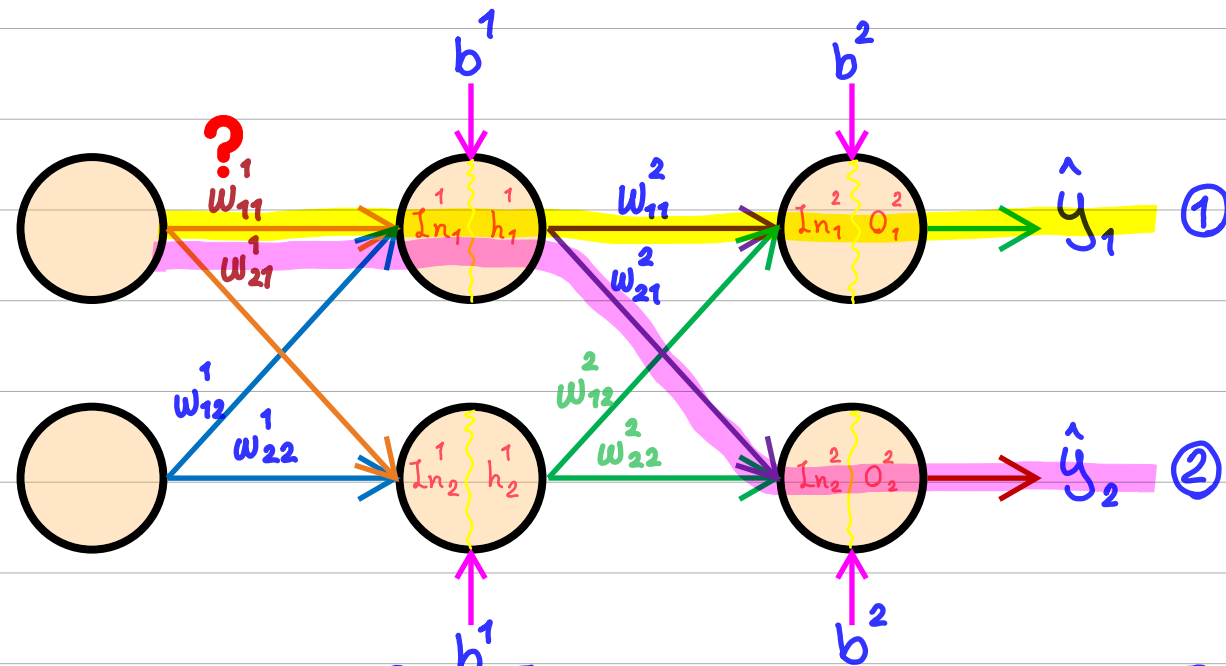
$$In_2^2 = W_{21}^2 h_1^1 + W_{22}^2 h_2^1 + b^2 = 1.2247 \rightarrow O_2^2 = \text{Sigmoid}(In_2^2) = \hat{y}_2$$

$$= 1.2247$$

$$= 0.7729$$

② Loss Function  $\rightarrow$   $MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$  SGD به ارای هر Sample که وارد شبکه می شود یکبار درخت را آپدیت می کند ✓

③ Back Propagation



$$\frac{\partial L}{\partial w_{11}^1} = \textcircled{1} + \textcircled{2} = \left[ \frac{\partial L}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial o_1^2} \frac{\partial o_1^2}{\partial \text{In}_1^2} \frac{\partial \text{In}_1^2}{\partial h_1^1} \frac{\partial h_1^1}{\partial \text{In}_1^1} \frac{\partial \text{In}_1^1}{\partial w_{11}^1} \right] + \left[ \frac{\partial L}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial o_2^2} \frac{\partial o_2^2}{\partial \text{In}_2^2} \frac{\partial \text{In}_2^2}{\partial h_1^1} \frac{\partial h_1^1}{\partial \text{In}_1^1} \frac{\partial \text{In}_1^1}{\partial w_{11}^1} \right]$$

$$\frac{\partial L}{\partial w_{11}^1} = \left[ \frac{\partial L}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial o_1^2} \frac{\partial o_1^2}{\partial \text{In}_1^2} \frac{\partial \text{In}_1^2}{\partial h_1^1} + \frac{\partial L}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial o_2^2} \frac{\partial o_2^2}{\partial \text{In}_2^2} \frac{\partial \text{In}_2^2}{\partial h_1^1} \right] \left[ \frac{\partial h_1^1}{\partial \text{In}_1^1} \frac{\partial \text{In}_1^1}{\partial w_{11}^1} \right]$$

$$* (a \cdot b + b \cdot c) = b(a + c)$$

$$\frac{\partial L}{\partial w_{11}^1} = \left[ \frac{\partial L}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial o_1^2} \frac{\partial o_1^2}{\partial \ln_1^2} \frac{\partial \ln_1^2}{\partial h_1^1} + \frac{\partial L}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial o_2^2} \frac{\partial o_2^2}{\partial \ln_2^2} \frac{\partial \ln_2^2}{\partial h_1^1} \right] \begin{bmatrix} \frac{\partial h_1^1}{\partial \ln_1^1} & \frac{\partial \ln_1^1}{\partial w_{11}^1} \end{bmatrix}$$

① Feed Forward:

$$\ln_1^1 = w_{11}^1 x_1 + w_{12}^1 x_2 + b^1 \leadsto h_1^1 = \text{ReLU}(\ln_1^1)$$

$$\ln_2^1 = w_{21}^1 x_1 + w_{22}^1 x_2 + b^1 \leadsto h_2^1 = \text{ReLU}(\ln_2^1)$$

$$\ln_1^2 = w_{11}^2 h_1^1 + w_{12}^2 h_2^1 + b^2 \leadsto o_1^2 = \text{Sigmoid}(\ln_1^2) = \hat{y}_1$$

$$\ln_2^2 = w_{21}^2 h_1^1 + w_{22}^2 h_2^1 + b^2 \leadsto o_2^2 = \text{Sigmoid}(\ln_2^2) = \hat{y}_2$$

$$\frac{\partial \ln_1^1}{\partial w_{11}^1} = \frac{\partial}{\partial w_{11}^1} (w_{11}^1 x_1 + \overset{0}{\cancel{w_{12}^1 x_2}} + \overset{0}{\cancel{b^1}}) = x_1$$

$$\frac{\partial \ln_1^2}{\partial h_1^1} = \frac{\partial}{\partial h_1^1} (w_{11}^2 h_1^1 + \overset{0}{\cancel{w_{12}^2 h_2^1}} + \overset{0}{\cancel{b^2}}) = w_{11}^2$$

$$\frac{\partial h_1^1}{\partial \ln_1^1} = \frac{\partial}{\partial \ln_1^1} (\text{ReLU}(\ln_1^1)) = \begin{cases} 1 & \ln_1^1 > 0 \\ 0 & \ln_1^1 < 0 \\ \text{Undefined} & \ln_1^1 = 0 \end{cases} \quad * \text{ReLU}(x) = \begin{cases} x & x > 0 \\ 0 & x < 0 \end{cases}$$

$$\frac{\partial o_1^2}{\partial \ln_1^2} = \frac{\partial}{\partial \ln_1^2} (\text{Sigmoid}(\ln_1^2)) = \sigma'(\ln_1^2) = \sigma(\ln_1^2) \times (1 - \sigma(\ln_1^2))$$

$$\frac{\partial \ln_2^2}{\partial h_1^1} = \frac{\partial}{\partial h_1^1} (w_{21}^2 h_1^1 + \overset{0}{\cancel{w_{22}^2 h_2^1}} + \overset{0}{\cancel{b^2}}) = w_{21}^2$$

$$\frac{\partial \hat{y}_1}{\partial o_1^2} = 1$$

$$\frac{\partial o_2^2}{\partial \ln_2^2} = \frac{\partial}{\partial \ln_2^2} (\text{Sigmoid}(\ln_2^2)) = \sigma'(\ln_2^2) = \sigma(\ln_2^2) \times (1 - \sigma(\ln_2^2))$$

$$\frac{\partial L}{\partial \hat{y}_1} = \frac{\partial}{\partial \hat{y}_1} \left( \frac{1}{m} \sum (y - \hat{y})^2 \right) = -\frac{2}{m} \sum (y - \hat{y})$$

$$\frac{\partial \hat{y}_2}{\partial o_2^2} = 1$$

$$\frac{\partial L}{\partial \hat{y}_2} = \frac{\partial}{\partial \hat{y}_2} \left( \frac{1}{m} \sum (y - \hat{y})^2 \right) = -\frac{2}{m} \sum (y - \hat{y})$$

\*  $(f^2)' = f \cdot f'$

$$\frac{\partial L}{\partial w_{11}^1} = \left[ \frac{\partial L}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial o_1^2} \frac{\partial o_1^2}{\partial \ln_1^2} \frac{\partial \ln_1^2}{\partial h_1^1} + \frac{\partial L}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial o_2^2} \frac{\partial o_2^2}{\partial \ln_2^2} \frac{\partial \ln_2^2}{\partial h_1^1} \right] \begin{bmatrix} \frac{\partial h_1^1}{\partial \ln_1^1} & \frac{\partial \ln_1^1}{\partial w_{11}^1} \end{bmatrix} \approx -0.0006647$$

$-0.249 \quad 1 \quad 0.187 \quad 0.4 \quad -0.217 \quad 1 \quad 0.174 \quad 0.55 \quad 0.3375 \quad 0.05$

$$\frac{\partial \ln_1^1}{\partial w_{11}^1} = \frac{\partial}{\partial w_{11}^1} (w_{11}^1 x_1 + \overset{0}{w_{12}^1} x_2 + b^1) = x_1 = 0.05$$

$$\frac{\partial \ln_1^2}{\partial h_1^1} = \frac{\partial}{\partial h_1^1} (w_{11}^2 h_1^1 + \overset{0}{w_{12}^2} h_2^1 + b^2) = w_{11}^2 = 0.4$$

$$= 0.187$$

$$\frac{\partial h_1^1}{\partial \ln_1^1} = \frac{\partial}{\partial \ln_1^1} (\text{ReLU}(\ln_1^1)) = \begin{cases} 1 & \ln_1^1 > 0 \\ 0 & \ln_1^1 < 0 \\ \text{Undefined} & \ln_1^1 = 0 \end{cases} = 0.3375$$

$$\frac{\partial o_1^2}{\partial \ln_1^2} = \frac{\partial}{\partial \ln_1^2} (\text{Sigmoid}(\ln_1^2)) = \sigma(\ln_1^2) \times (1 - \sigma(\ln_1^2))$$

$$\frac{\partial \ln_2^2}{\partial h_1^1} = \frac{\partial}{\partial h_1^1} (w_{21}^2 h_1^1 + \overset{0}{w_{22}^2} h_2^1 + b^2) = w_{21}^2 = 0.55$$

$$\frac{\partial \hat{y}_1}{\partial o_1^2} = 1$$

$$= -0.249$$

$$\frac{\partial o_2^2}{\partial \ln_2^2} = \frac{\partial}{\partial \ln_2^2} (\text{Sigmoid}(\ln_2^2)) = \sigma(\ln_2^2) \times (1 - \sigma(\ln_2^2)) = 0.174$$

$$\frac{\partial L}{\partial \hat{y}_1} = \frac{\partial}{\partial \hat{y}_2} \left( \frac{1}{m} \sum (y - \hat{y})^2 \right) = -\frac{2}{m} \sum (y - \hat{y})$$

$$\frac{\partial \hat{y}_2}{\partial o_2^2} = 1$$

$$\frac{\partial L}{\partial \hat{y}_2} = \frac{\partial}{\partial \hat{y}_2} \left( \frac{1}{m} \sum (y - \hat{y})^2 \right) = -\frac{2}{m} \sum (y - \hat{y}) = -0.217$$

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial L}{\partial w_{\text{old}}}$$

$$w_{11}^1_{\text{new}} = 0.15 - (0.1 \times (-0.0006647)) = 0.15006647$$

# Gradient Descent

## Stochastic Gradient Descent (SGD)

در این روش، گرادیان برای هر نمونه داده به صورت جداگانه محاسبه می‌شود

$$W_{new} = W_{old} - \alpha \frac{\partial L}{\partial W_{old}}$$

## Batch Gradient Descent

در این روش، گرادیان کل مجموعه داده برای هر گام محاسبه می‌شود

## Mini-batch Gradient Descent

این روش ترکیبی از دو روش قبلی است، به این صورت که گرادیان برای دسته‌های کوچکی از داده‌ها (مینی‌بچ‌ها) محاسبه می‌شود

Epoch

یک دوره زمانی است که الگوریتم یادگیری ماشین یک بار کامل بر روی کل مجموعه داده‌ها آموزش می‌بیند

Batch

یک بچ مجموعه‌ای از نمونه‌های داده است که در یک مرحله از الگوریتم گرادیان دیسنت استفاده می‌شود

# Stochastic Gradient Descent (SGD)

```
# Set number of epochs and constant learning rate
n_epochs = 50
alpha = 0.1

# Initialize parameters randomly
w = np.random.randn(2,1)

# Training loop
for epoch in range(n_epochs):
    for i in range(m): # m should be defined as the number of samples in the dataset
        random_index = np.random.randint(m) # Select a random index
        xi = X_b[random_index:random_index+1] # Extract features for selected sample
        yi = y[random_index:random_index+1] # Extract target for selected sample

        # Compute gradient of loss function
        gradients = 2 * xi.T.dot(xi.dot(w) - yi)

        # Update parameters
        w = w - alpha * gradients # Apply gradient descent step
```



# Batch Gradient Descent

```
n_epochs = 50
alpha = 0.1 # learning rate

w = np.random.randn(2,1) # random initialization

for epoch in range(n_epochs):
    gradients = 2/m * X_b.T.dot(X_b.dot(w) - y)
    w = w - alpha * gradients
```

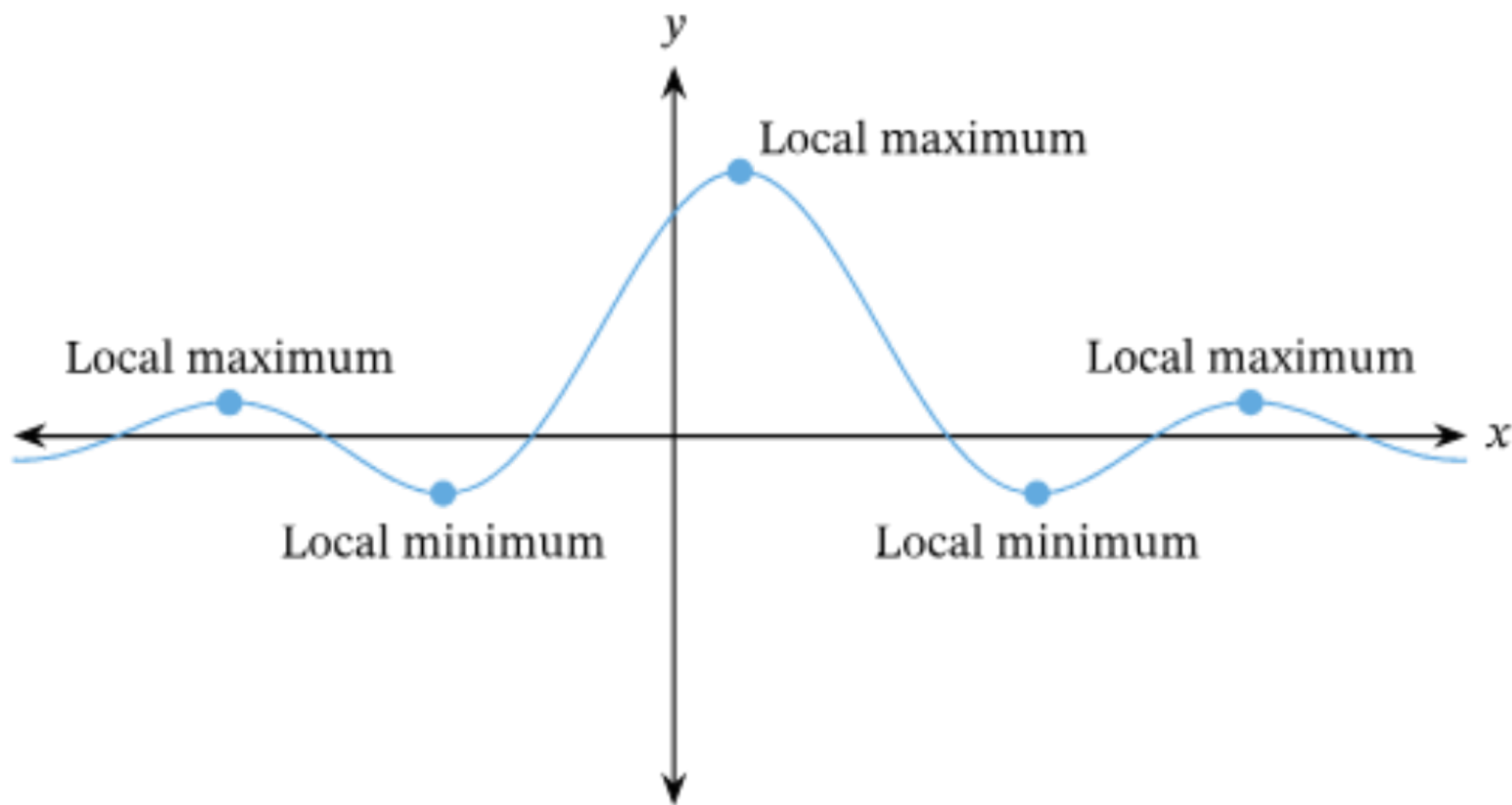
# Mini-batch Gradient Descent

```
n_epochs = 50
batch_size = 20 # size of mini-batch
alpha = 0.1 # learning rate

w = np.random.randn(2,1) # random initialization

for epoch in range(n_epochs):
    shuffled_indices = np.random.permutation(m)
    X_b_shuffled = X_b[shuffled_indices]
    y_shuffled = y[shuffled_indices]
    for i in range(0, m, batch_size):
        xi = X_b_shuffled[i:i+batch_size]
        yi = y_shuffled[i:i+batch_size]
        gradients = 2/batch_size * xi.T.dot(xi.dot(w) - yi)
        w = w - alpha * gradients
```

| Mini-batch Gradient Descent     | Stochastic Gradient Descent (SGD)                    | Batch Gradient Descent           | معیار مقایسه           |
|---------------------------------|------------------------------------------------------|----------------------------------|------------------------|
| دسته‌های کوچکی از داده‌ها       | هر نمونه داده به صورت جداگانه                        | تمام داده‌ها                     | محاسبه گرادیان         |
| تعادلی بین سرعت و دقت           | سریع                                                 | کند                              | سرعت                   |
| کمتر از Batch و بیشتر از SGD    | کم                                                   | زیاد                             | نیاز به حافظه          |
| تعادلی بین نویز و دقت           | نویز بالا                                            | بالا                             | دقت گرادیان            |
| کمتر از SGD و بیشتر از Batch    | زیاد                                                 | کم                               | نوسانات در جهت گرادیان |
| تعادل بین همگرایی و نوسانات     | نوسانات زیاد می‌تواند از مینیمم‌های محلی جلوگیری کند | ممکن است به مینیمم‌های محلی برسد | همگرایی                |
| پیچیده‌تر از دو روش دیگر        | ساده                                                 | ساده                             | پیاده‌سازی             |
| مناسب برای اکثر مسائل و داده‌ها | مناسب برای داده‌های بسیار بزرگ                       | مناسب برای داده‌های کوچک         | کاربرد                 |
| بین Batch و SGD                 | کم                                                   | زیاد                             | زمان‌بری               |



\* Non-Convex

# چالش‌های Gradient Descent

## Local Minimum

جایی که الگوریتم ممکن است در یک نقطه حداقل محلی گیر بیفتد که پایین‌ترین نقطه در تابع هزینه نیست

## Global Minimum

جایی که تابع هزینه به پایین‌ترین مقدار خود می‌رسد و هدف نهایی الگوریتم است

## Plateau

ناحیه‌ای که در آن تابع هزینه تغییرات کمی دارد و الگوریتم ممکن است به کندی حرکت کند یا گیر بیفتد

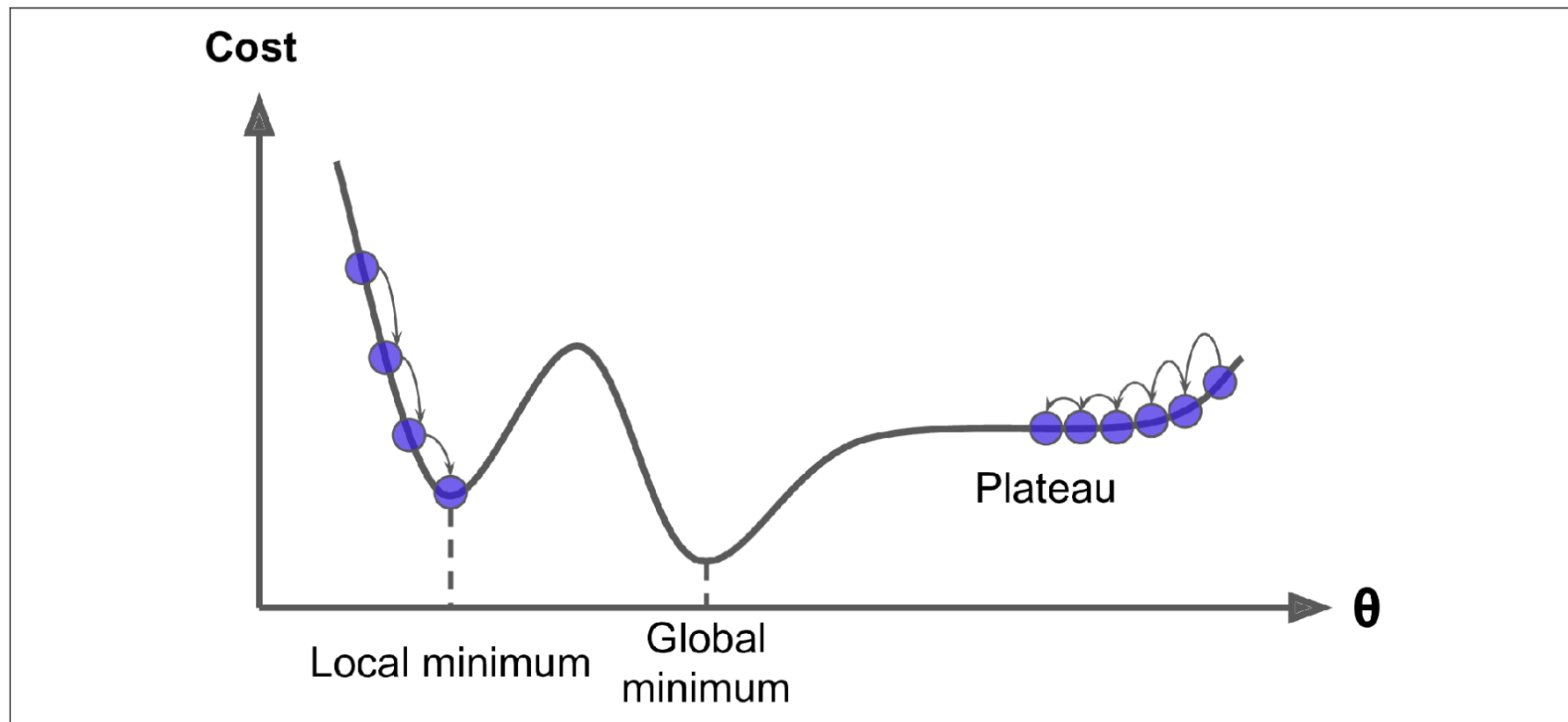


Figure 4-6. Gradient Descent pitfalls

| مشکل/چالش                          | توضیح                                                                                                    | راه حل‌ها                                                                                                                                              |
|------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| گیر کردن در نقاط محلی              | ممکن است گرادیان کاهشی در نقاط محلی به جای نقطه‌ای که بهینه‌ترین است گیر کند.                            | استفاده از الگوریتم‌های بهینه‌سازی مانند Momentum, Nesterov Accelerated Gradient یا استفاده از شبکه‌های عصبی عمیق با ساختارهای متفاوت.                 |
| انتخاب نرخ یادگیری (Learning Rate) | انتخاب نرخ یادگیری نهایی یک چالش است که می‌تواند منجر به سرعت یادگیری ناکافی یا رفتار نامناسب دیگری شود. | استفاده از روش‌های تطبیقی نرخ یادگیری مانند AdaGrad, RMSprop, Adam و تنظیم نرخ یادگیری به صورت دینامیک.                                                |
| سختی تنظیم پارامترها               | برای مسائل با تعداد زیادی پارامتر، تنظیم نرخ یادگیری و سایر پارامترها ممکن است مشکل‌ساز باشد.            | استفاده از روش‌های بهینه‌سازی خودکار و توسعه یافته‌تر مانند hyperparameter tuning با استفاده از روش‌های بهینه‌سازی مانند Grid Search یا Random Search. |
| مقیاس‌پذیری                        | برای داده‌های بزرگ، استفاده از تمام داده‌ها در هر مرحله ممکن نیست و می‌تواند باعث ابتلای گرادیان شود.    | استفاده از روش‌های کاهش ابعاد مانند Mini-Batch Gradient Descent و استفاده از تکنیک‌های توزیع شده برای پردازش موازی داده.                               |
| حساسیت به مقدار اولیه              | نتیجه نهایی ممکن است به شدت به مقدار اولیه وابسته باشد، به خصوص در شبکه‌های عصبی.                        | استفاده از روش‌های بهینه‌سازی با اولویت برای تنظیم وابستگی کمتر به مقدار اولیه مانند استفاده از توابع فعال‌سازی و پیش‌پردازش مناسب.                    |
| نقاط مستعد برای ارتفاع خطا         | در برخی مسائل، ممکن است گرادیان کم یا ناچیز باشد، که به تدریج باعث کاهش کارایی می‌شود.                   | استفاده از روش‌های بهینه‌سازی خاص مانند Batch Normalization, Dropout و استفاده از روش‌های پیشرفته‌تر برای پیش‌پردازش داده.                             |

# مدیریت چالش های گرادیان کاهشی

Scaling مقیاس بندی

(Learning Rate) استفاده از نرخ یادگیری مناسب

استفاده از روش های پیشرفته تری مانند

Adam و RMSprop، AdaGrad

(Momentum) اضافه کردن مومنتوم

(Random Initialization) ابتدا قرار دادن وزن ها در نقاط مختلف

# scaling ----> Normalization

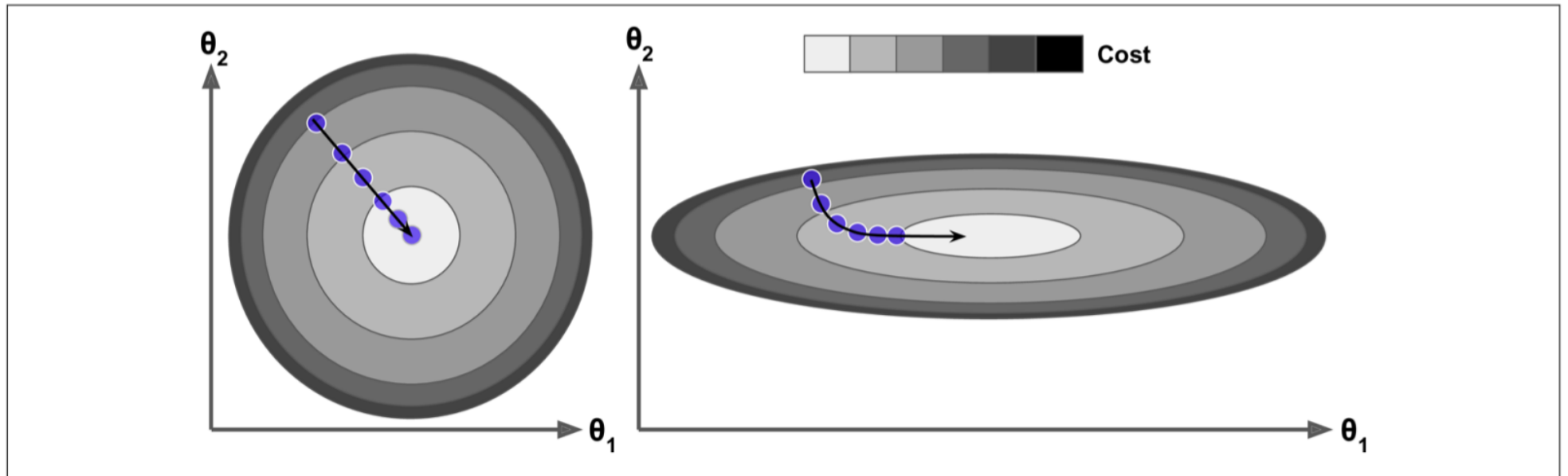


Figure 4-7. Gradient Descent with (left) and without (right) feature scaling



# مومنتوم یا Momentum

$$W_{\text{new}} = W_{\text{old}} - \alpha \frac{\partial L}{\partial W_{\text{old}}}$$

$$\left. \begin{array}{l} \text{Adding Momentum} \rightarrow W_{\text{new}} = W_{\text{old}} + m \\ m = \beta m - \alpha \frac{\partial L}{\partial W_{\text{old}}} \end{array} \right\} \rightarrow \begin{array}{l} W^{t+1} = W^t + m^{t+1} \\ m^{t+1} = \beta_1 m^t - \alpha \frac{\partial L}{\partial W^t} \end{array}$$

  $\alpha = 0.1 \quad \beta = 0.9 \quad W^0 = 1$

1. Initialize  $\rightarrow W^0 = 1 \quad m^0 = 0$

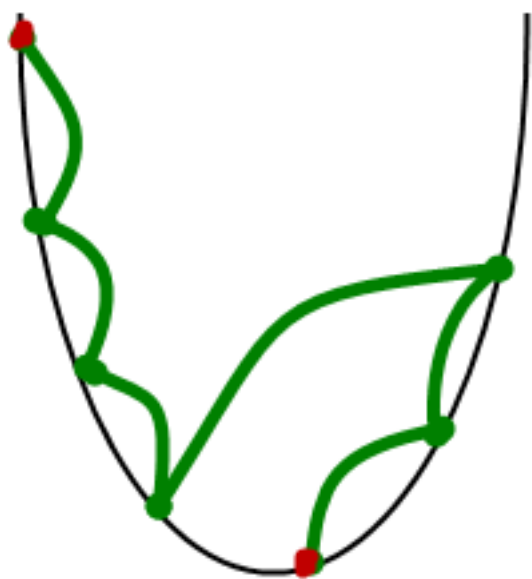
2. First Iter

Assume  $\frac{\partial L}{\partial W} = 2$

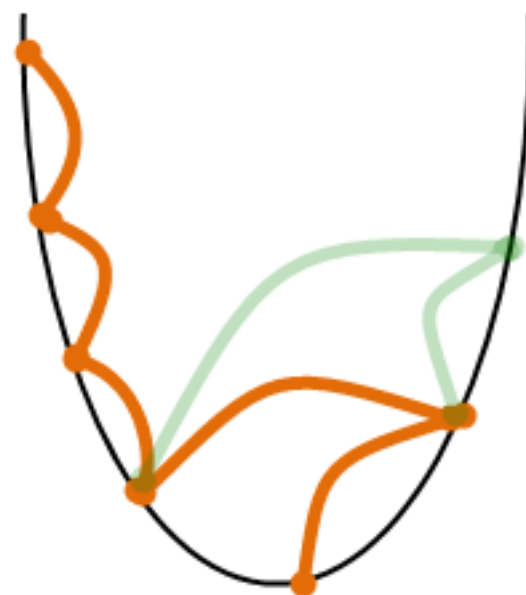
Update  $m^1 \rightarrow m^1 := \beta_1 m^0 - \alpha \frac{\partial L}{\partial W^0} = -0.2$

$$W^1 = W^0 + m^{t+1} = 0.8$$

Momentum



Nesterov Momentum



# Nesterov Momentum

$$\begin{array}{l}
 \text{Momentum} \rightsquigarrow w^{t+1} := w^t + m^{t+1} \\
 m^{t+1} := \beta_1 m^t - \alpha \frac{\partial L}{\partial w^t}
 \end{array}
 \left. \vphantom{\begin{array}{l} \text{Momentum} \rightsquigarrow w^{t+1} := w^t + m^{t+1} \\ m^{t+1} := \beta_1 m^t - \alpha \frac{\partial L}{\partial w^t} \end{array}} \right\} \text{Nesterov} \rightarrow \begin{array}{l}
 \hat{w}^{t+1} := w^t + \beta_1 m^t \\
 m^{t+1} := \beta_1 m^t - \alpha \frac{\partial L}{\partial w^t}(\hat{w}^{t+1}) \\
 w^{t+1} := w^t + m^{t+1}
 \end{array}$$

  $\alpha = 0.1 \quad \beta = 0.9 \quad w^0 = 1$

1. Initialize  $\rightsquigarrow w^0 = 1 \quad m^0 = 0$

2. First Iter

$$\hat{w}^1 := \overset{1}{w^0} + \overset{0.9}{\beta_1 m^0} = 1$$

Assume  $\frac{\partial L}{\partial w}(\hat{w}^1) = 2$

$$m^1 := \overset{0.9}{\beta_1 m^0} - \overset{0.1}{\alpha} \frac{\partial L}{\partial w^t}(\hat{w}^{t+1}) = 0.9 - 0.2 = 0.7$$

$$w^1 := \overset{1}{w^0} + \overset{-0.2}{m^1} = 0.8$$

| ویژگی            | گرادیان با مومنتوم (Gradient with Momentum)  | نستروف (- Nesterov Accelerated Gradient (NAG) |
|------------------|----------------------------------------------|-----------------------------------------------|
| نحوه بروزرسانی   | سرعت به روزرسانی تجمعی از گرادیان‌های قبلی   | گرادیان از نقطه پیش‌بینی شده (lookahead)      |
| مزایا            | کاهش نوسانات در راستای عمود بر جهت گرادیان   | پیش‌بینی بهتری از گرادیان‌های آینده           |
| معایب            | ممکن است به راحتی در مینیمم‌های محلی گیر کند | ممکن است محاسبات بیشتری نیاز داشته باشد       |
| سرعت همگرایی     | سریع‌تر از گرادیان نزولی ساده                | سریع‌تر از مومنتوم استاندارد                  |
| پیچیدگی محاسباتی | متوسط                                        | نسبتاً بالا                                   |
| پارامترها        | نرخ یادگیری $(\eta)$ ، مومنتوم $(\beta)$     | نرخ یادگیری $(\eta)$ ، مومنتوم $(\beta)$      |

# پارامترها

پارامترها مقادیر داخلی یک مدل یادگیری ماشین هستند که در طی فرایند آموزش مدل از داده‌ها، یاد گرفته (تنظیم) می‌شوند. این پارامترها معمولاً وزن‌ها و بایاس‌ها در مدل‌های شبکه عصبی یا ضرایب در مدل‌های رگرسیون هستند. پارامترها مستقیماً از داده‌های آموزشی استخراج می‌شوند و مقادیرشان به طور خودکار تنظیم می‌شود تا مدل بتواند عملکرد بهتری روی داده‌های آموزشی داشته باشد

وزن‌ها در شبکه‌های عصبی (weights)

بایاس‌ها در شبکه‌های عصبی (biases)

# هایپرپارامترها

هایپرپارامترها مقادیری هستند که قبل از فرایند آموزش مدل تعیین می‌شوند و بر روی فرایند آموزش و ساختار مدل تأثیر می‌گذارند. هایپرپارامترها توسط انسان یا به کمک روش‌های جستجوی هایپرپارامتر مانند گرید سرچ یا جستجوی تصادفی تنظیم می‌شوند. تنظیم درست هایپرپارامترها می‌تواند تأثیر بزرگی بر عملکرد نهایی مدل داشته باشد

مثال‌هایی از هایپرپارامترها

نرخ یادگیری (learningrate)

تعداد لایه‌ها و نوروها در هر لایه (numberoflayers و numberofneuronsperlayer)

اندازه بچ، تعداد نمونه‌هایی که قبل از به‌روزرسانی وزن‌ها در طول یک مرحله (batchsize) آموزش پردازش می‌شوند

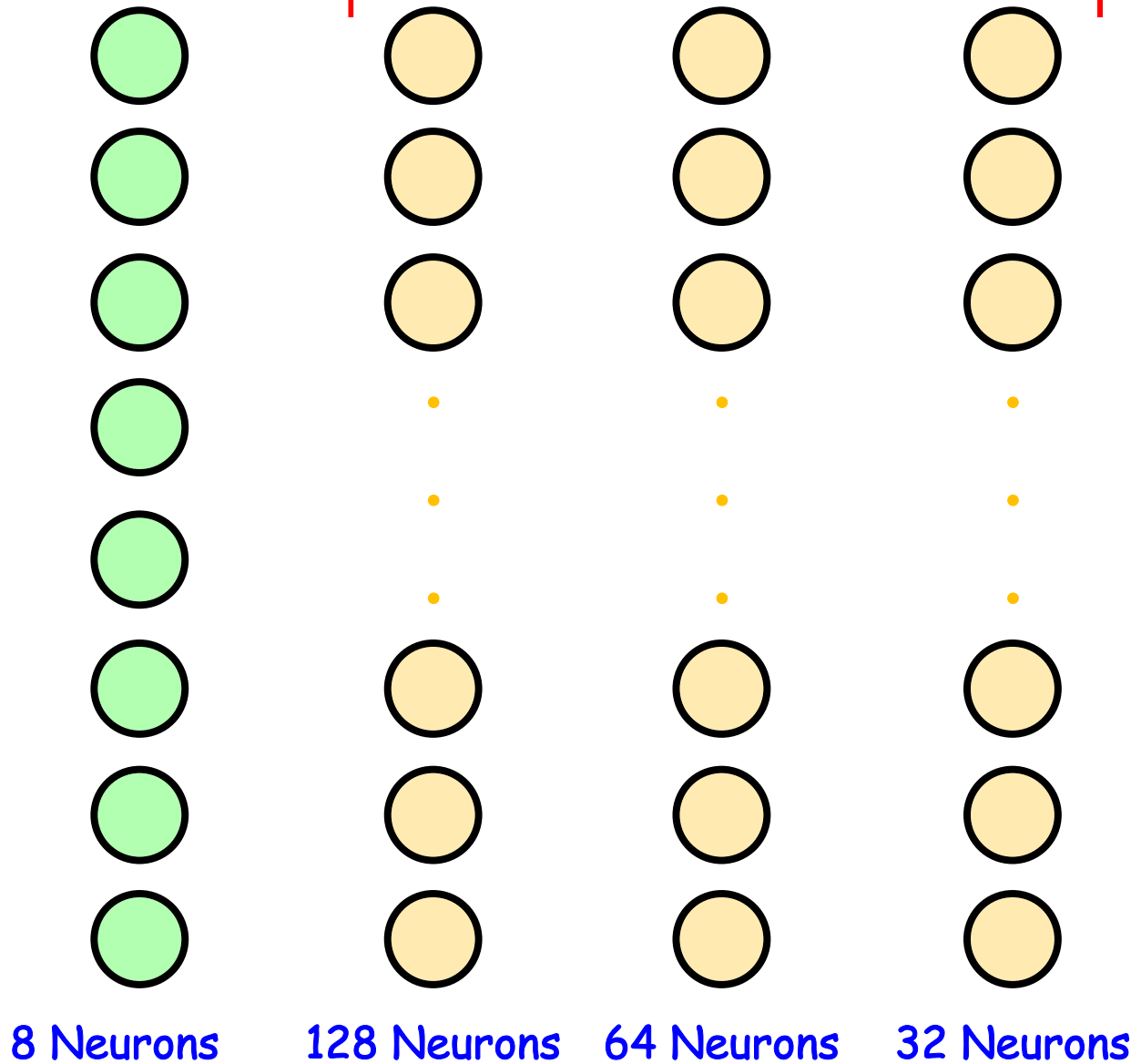
تعداد دوره‌های آموزش، تعداد تکرارهای کامل روی داده‌های آموزشی در طول آموزش (numberofepochs)

ساختار مدلی که در کد خواهید دید

Input Layer

Hidden Layer

Output Layer



1 Neurons

\* fully connected layer

## Some Sources:

<https://medium.com/data-science-365/overview-of-a-neural-networks-learning-process-61690a502fa>

<https://www.javatpoint.com/pytorch-backpropagation-process-in-deep-neural-network>

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow

<https://towardsdatascience.com/gradient-descent-algorithm-a-deep-dive-cf04e8115f21>

<https://www.nagwa.com/en/explainers/597191262989/>

[https://gbhat.com/machine\\_learning/gradient\\_descent\\_nesterov.html](https://gbhat.com/machine_learning/gradient_descent_nesterov.html)

<https://devskrol.com/2020/09/05/optimizer-in-neural-network/>

<https://towardsdatascience.com/learning-parameters-part-2-a190bef2d12>